Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Manejo e Implementacion de Archivos

Sección: A

“Manual Tecnico Proyecto #2”

Estudiante:

Mynor Migue Monzón Martínez 202230884

Quetzaltenango, 24 de octubre de 2026

Introducción

El presente manual técnico documenta el diseño y la implementación del Sistema de Gestión Documental, una plataforma desarrollada para una organización dedicada a la prestación de servicios profesionales, que permite almacenar, organizar, consultar y compartir archivos de texto (.txt) y Markdown (.md) entre sus usuarios. El documento está dirigido al personal técnico encargado de revisar, mantener o ampliar el sistema, y complementa al Manual de Usuario, que describe el uso funcional de la aplicación.

El sistema integra la administración de usuarios, carpetas jerárquicas, archivos cuyo contenido se guarda dentro de la propia base de datos mediante GridFS, compartición con tres niveles de permiso, comentarios, una papelera con restauración, un historial de actividad y un módulo de reportes basado en agregaciones de MongoDB.

1. Arquitectura del Sistema

1.1 Arquitectura cliente-servidor

El sistema está compuesto por dos aplicaciones independientes que se comunican mediante una API REST sobre HTTP:

- **Aplicación cliente:** desarrollada en Angular 21, se ejecuta en el navegador del usuario y consume la API mediante peticiones HTTP con formato JSON.
- **Aplicación servidor:** desarrollada en Node.js con Express, expone la API REST y es la única responsable de comunicarse con la base de datos MongoDB. El cliente nunca accede a la base de datos de forma directa.

La base de datos MongoDB es de uso exclusivo del servidor. Esta separación permite que ambas aplicaciones evolucionen y se desplieguen de forma independiente.

1.2 Arquitectura en capas del backend

Dentro del servidor, el código (carpeta `server/src`) se organiza en capas con responsabilidades separadas:

Capa	Carpeta	Responsabilidad
Configuración	`config`	Lee el archivo `.env` y exige que exista `JWT_SECRET`; conecta con MongoDB mediante Mongoose y crea el bucket de GridFS.
Rutas	`routes`	Declaran los endpoints de cada módulo y les asignan los middlewares de autenticación, de rol y de subida de archivos.
Controladores	`controllers`	Reciben la petición, validan los datos de entrada, aplican las reglas del módulo y construyen la respuesta JSON.
Servicios	`services`	Lógica compartida por varios controladores: permisos de acceso, nombres únicos, GridFS, eliminación definitiva, historial de actividad y reportes.

Capa	Carpeta	Responsabilidad
Modelos	`models`	Esquemas de Mongoose que representan las colecciones de la base de datos.
Middlewares	`middlewares`	Autenticación con JWT, control de acceso por rol y manejo global de errores.
Utilidades	`utils`	Funciones de apoyo: errores con código HTTP, validación de nombres y archivos, fechas, parámetros y serialización de respuestas.

Una petición típica recorre las capas de la siguiente manera: Express la recibe en `app.js`; la ruta correspondiente ejecuta primero el middleware `authenticate` (y `requireRole` cuando el módulo es solo para administradores); el controlador valida los datos y consulta los permisos con `access.service`; luego lee o escribe en MongoDB mediante los modelos o los servicios y registra la operación en el historial; finalmente responde con JSON. Cualquier error llega al manejador global, que siempre responde con la forma `{ "message": "..." }`. Los controladores no contienen detalles de GridFS ni el cálculo de permisos: esos detalles viven en los servicios.

2. Tecnologías y Herramientas

Componente	Tecnología	Versión
Entorno de ejecución del servidor	Node.js	22
Framework del servidor	Express	5.x
Base de datos	MongoDB	(completar con la versión instalada)
Acceso a la base de datos	Mongoose	9.x
Almacenamiento de archivos	GridFS (GridFSBucket)	Incluido en el driver de MongoDB
Autenticación	jsonwebtoken (JWT)	9.x
Cifrado de contraseñas	bcryptjs	3.x
Subida de archivos	multer	2.x
Configuración y CORS	dotenv y cors	18.x y 2.x
Reinicio automático en desarrollo	nodemon	3.x
Scripts de carga inicial	Node.js con el driver mongodb y bcryptjs	7.x y 3.x

Componente	Tecnología	Versión
Framework del cliente	Angular	21
Lenguaje del cliente	TypeScript	5.9
Programación reactiva	RxJS	7.8
Vista de archivos Markdown	marked	18.x
Control de versiones	Git / GitHub	—

Las versiones exactas de cada dependencia se encuentran en los archivos `package.json` y `package-lock.json` de las carpetas server, client y database. Como herramientas de desarrollo se utilizaron Visual Studio Code, MongoDB Compass como cliente gráfico de MongoDB y draw.io para los diagramas.

3. Estructura del Repositorio

El repositorio se organiza en cuatro carpetas principales, correspondientes a las dos aplicaciones, los scripts de la base de datos y la documentación:

```
Proyecto_2_Manejo_De_Archivos_Mynor_Monzon_2S_2026/
|-- server/                    Aplicacion servidor (Node.js + Express)
|   |-- src/
|   |   |-- config/            Variables de entorno y conexion a MongoDB
|   |   |-- controllers/       Un controlador por modulo
|   |   |-- middlewares/       Autenticacion, roles y errores
|   |   |-- models/            Esquemas de Mongoose
|   |   |-- routes/            Rutas de la API
|   |   |-- services/          Permisos, GridFS, papelera, reportes
|   |   |-- utils/             Funciones de apoyo
|   |   |-- app.js             Configuracion de Express
|   |   `-- server.js          Arranque del servidor
|   |-- .env.example           Plantilla de configuracion
|   `-- package.json
|-- client/                    Aplicacion cliente (Angular 21)
|   `-- src/app/
|       |-- core/              Modelos, servicios, guards e interceptores
|       |-- shared/            Componentes comunes
|       |-- layout/            Menu lateral
|       `-- features/          Una carpeta por pantalla
|-- database/                  Scripts de carga inicial
|   |-- indexes.js             Crea los indices
|   `-- seed.js                Carga datos iniciales y GridFS
`-- docs/                      Manuales, modelo documental y diagramas
```

4. Modelo de Datos

4.1 Modelo documental

La base de datos `gestion_documental` está formada por seis colecciones propias y por las dos colecciones que crea GridFS. El diseño completo, con ejemplos de cada documento y la justificación de lo embebido y lo referenciado, se encuentra en el archivo `docs/modelo-documental.md`; esta sección resume lo esencial.

Colección	Descripción	Referencias
`users`	Cuentas del sistema. Guarda el rol, el estado activo o inactivo, el espacio usado y el hash bcrypt de la contraseña.	—
`folders`	Carpetas y subcarpetas. Guarda su carpeta padre, el arreglo embebido `ancestors` y la fecha de eliminación (`deletedAt`) para la papelera.	ownerId → users, parentId → folders
`files`	Datos de cada archivo (nombre, tipo, tamaño, fechas) y la referencia a su contenido.	ownerId → users, folderId → folders, gridfsId → fs.files
`fs.files` y `fs.chunks`	Contenido de los archivos, guardado en fragmentos de 255 KB por GridFS.	—
`shares`	Permisos otorgados sobre archivos y carpetas: quién comparte, con quién y con qué nivel.	ownerId → users, sharedWithId → users, resourceId → files o folders
`comments`	Comentarios de los archivos.	fileId → files, userId → users
`activity_logs`	Historial de operaciones. Guarda una copia del nombre del recurso para que siga siendo legible aunque se elimine.	userId → users, resourceId → files, folders o users

4.2 Índices

Los índices se crean con el script `database/indexes.js`; en los esquemas de Mongoose se desactiva la creación automática (`autoIndex: false`) para que la base de datos dependa únicamente de ese script.

Colección	Índices	Para qué sirven
`users`	`email` (único)	Inicio de sesión
`folders`	`ownerId, parentId, deletedAt; ancestors`	Listar contenido, papelera, subcarpetas

Colección	Índices	Para qué sirven
`files`	`ownerId, folderId, deletedAt`; `ownerId, createdAt`; `ext`; `gridfsId`; `name` (texto)	Listados, reportes y búsqueda
`shares`	`sharedWithId, createdAt`; `ownerId, createdAt`; `resourceType, resourceId`; `resourceType, resourceId, sharedWithId` (único)	Permisos y reportes de compartición
`comments`	`fileId, createdAt`	Comentarios de un archivo
`activity_logs`	`userId, createdAt`; `resourceType, resourceId`; `action`; `createdAt`	Historial y reportes de actividad

4.3 Carga inicial con scripts

El enunciado exige que la base de datos pueda inicializarse con información suficiente para la calificación. Esto se resuelve con dos scripts de Node.js en la carpeta database, que leen la configuración del mismo archivo `server/.env` que usa el servidor:

- `indexes.js`: crea la base de datos y todos los índices de las seis colecciones.
- `seed.js`: borra los datos existentes (sin tocar los índices) y carga 3 usuarios, 14 carpetas y subcarpetas, 50 archivos con su contenido guardado en GridFS, 33 comparticiones, comentarios, elementos en la papelera y cientos de registros de historial.

El `seed.js` usa un generador pseudoaleatorio con semilla fija, de modo que siempre crea los mismos datos, y cifra las contraseñas con bcrypt igual que el sistema. Las fechas se calculan en relación con el momento en que se ejecuta, para que los reportes por rango de fechas tengan información reciente.

5. Seguridad

5.1 Autenticación con JWT

El sistema utiliza JSON Web Tokens (JWT) y no mantiene sesiones en el servidor. El flujo es el siguiente:

1. El usuario envía su correo y contraseña a `POST /api/auth/login`.
2. El controlador busca al usuario por correo y compara la contraseña recibida con el hash bcrypt guardado. Si no coincide, responde 401 con el mismo mensaje tanto para un correo inexistente como para una contraseña incorrecta, para no revelar qué correos existen. Si la cuenta está inactiva, responde 403.
3. Si las credenciales son correctas, se firma un token con el secreto `JWT_SECRET`, que lleva el id del usuario como sujeto, el rol y una expiración de 8 horas (`JWT_EXPIRES_IN`). Además se registra `auth.login` en el historial.
4. El cliente guarda el token y lo envía en cada petición posterior con el encabezado `Authorization: Bearer <token>`.
5. En cada petición, el middleware `authenticate` valida la firma y la vigencia del token y vuelve a consultar al usuario en la base de datos: si fue eliminado o desactivado después de emitir el token, la petición se rechaza con 401. Por eso desactivar una cuenta bloquea su acceso de inmediato.

5.2 Autorización

Hay dos niveles de control de acceso:

- **Por rol:** el middleware `requireRole('admin')` limita a los administradores la administración de usuarios y los reportes globales. Un usuario sin el rol recibe 403.
- **Por recurso:** cada operación sobre una carpeta o un archivo consulta `access.service`, que determina el nivel del usuario sobre ese recurso (lectura, comentar, edición o propietario) y rechaza la acción si no alcanza el nivel requerido. Este control aplica a todos los roles: el administrador no tiene acceso especial al contenido de los archivos de otros usuarios.

5.3 Cifrado de contraseñas

Las contraseñas nunca se almacenan en texto plano. La librería bcryptjs genera un hash con sal aleatoria y factor de costo 10 al crear un usuario o cambiar su contraseña, y el mismo mecanismo se usa para verificarla en cada inicio de sesión. Las respuestas de la API nunca incluyen el hash: los datos del usuario pasan por una función de serialización que solo entrega los campos públicos.

5.4 Otras medidas

- **Validación de archivos:** solo se aceptan extensiones .txt y .md, con un máximo de 5 MB, y se rechaza el contenido que contenga bytes nulos (un binario renombrado como .txt). El contenido se recibe en memoria y se guarda en GridFS.
- **Nombres de carpetas y archivos:** se rechazan los símbolos `/` y `\`, los nombres vacíos o mayores de 100 caracteres.
- **CORS:** el servidor solo acepta peticiones del origen definido en `CLIENT_URL` (por defecto `http://localhost:4200`).
- **Errores:** el manejador global nunca envía detalles internos; los errores del servidor responden un mensaje genérico y el detalle solo queda en la consola.
- **Contenido Markdown:** al mostrarse en el cliente pasa por el sanitizador de Angular, que elimina scripts, eventos y enlaces peligrosos que pudiera traer un archivo escrito por otra persona.
- **Secretos:** el archivo `.env` no se sube al repositorio; se entrega una plantilla `.env.example` con valores de ejemplo.

6. Lógica de Negocio

6.1 Permisos y herencia

Los permisos otorgados se guardan en la colección `shares`. Los niveles son acumulativos y se ordenan así: lectura, comentar, edición y propietario (este último no se otorga: corresponde a quien creó el elemento). Al compartir una carpeta, el permiso alcanza a todo su contenido: para saber el nivel de un usuario sobre un recurso, `access.service` busca permisos sobre el propio recurso y sobre todas sus carpetas padre (el arreglo `ancestors`), y aplica el más alto. Para quien recibe una carpeta, la ruta de navegación empieza en la carpeta compartida: el sistema no muestra las carpetas que están por encima.

Las operaciones exigen un nivel mínimo: ver, descargar y listar requieren lectura; comentar requiere comentar; editar el contenido, renombrar, crear carpetas y subir archivos dentro de una carpeta compartida requieren edición; mover, eliminar y compartir son solo del propietario. Lo que una persona con permiso de edición crea o sube dentro de una carpeta compartida pertenece al propietario de esa carpeta, y el espacio se suma a su cuenta.

6.2 Carpetas jerárquicas

Cada carpeta guarda su carpeta padre y, de forma redundante, la lista completa de sus ancestros (`ancestors`, de la raíz hacia abajo). Esa redundancia permite resolver con una sola consulta tres problemas: detectar ciclos, heredar permisos y encontrar todas las subcarpetas de una carpeta. Al mover una carpeta, se rechaza el movimiento si el destino es la propia carpeta o aparece entre sus descendientes, y se recalcula `ancestors` en la carpeta y en todas sus subcarpetas con una sola

operación `bulkWrite`. Los archivos solo guardan su carpeta directa, por lo que no se modifican al mover una carpeta.

6.3 Archivos y GridFS

El contenido de cada archivo se guarda dentro de MongoDB mediante GridFS (colecciones `fs.files` y `fs.chunks`), y el documento de `files` conserva únicamente los datos del archivo y el identificador de su contenido (`gridfsId`). Esto permite listar y filtrar archivos sin leer su contenido. Los flujos principales son:

- **Subida:** se valida el nombre, la extensión y el contenido; se guarda el contenido en GridFS; se crea el registro en `files`; se suma el tamaño al espacio usado del propietario y se registra `file.upload`. Si falla la creación del registro, el contenido ya guardado se elimina.
- **Edición del contenido:** se guarda primero el contenido nuevo en GridFS, luego se actualiza `gridfsId` y el tamaño, y al final se borra el contenido anterior; el espacio usado se ajusta por la diferencia.
- **Descarga y lectura:** el contenido se envía directamente desde GridFS hacia la respuesta HTTP (descarga) o se lee completo y se devuelve como texto (visor).
- **Nombres:** no puede haber dos archivos (ni dos carpetas) con el mismo nombre en la misma ubicación, sin distinguir mayúsculas.

6.4 Papelera

Eliminar no borra: asigna a `deletedAt` la fecha actual, y todas las consultas normales filtran los elementos con `deletedAt` vacío. Al enviar una carpeta a la papelera, ella y todo lo que contiene (subcarpetas y archivos activos) reciben exactamente la misma fecha. Esa marca común permite restaurar la carpeta con lo que cayó junto con ella sin devolver lo que ya estaba en la papelera desde antes, y permite listar en la papelera solo lo eliminado directamente, mostrando lo demás dentro de su carpeta.

Al restaurar, si la carpeta original sigue en la papelera el elemento pasa a la raíz (y se recalcula su ruta); si el nombre ya está ocupado en el destino, se agrega el sufijo " (restaurado)". La eliminación definitiva borra el registro, luego el contenido en GridFS y después los permisos y comentarios asociados, y descuenta el espacio liberado; en las carpetas lo hace sobre todo el árbol. El historial no se toca.

6.5 Consistencia sin transacciones

MongoDB se ejecuta como una instancia única (sin conjunto de réplicas), y en ese modo no hay transacciones. Por eso las operaciones de varios pasos se ordenan para que un fallo a la mitad deje siempre un estado válido y recuperable: al enviar una carpeta a la papelera se marcan primero los archivos, luego las subcarpetas y al final la carpeta; al restaurar se hace en orden inverso; al eliminar definitivamente se borra primero el registro y después el contenido, de modo que lo peor

que puede quedar es un fragmento sin dueño en GridFS y nunca un archivo que apunte a un contenido inexistente.

6.6 Historial de actividad

Cada operación relevante (inicio de sesión, carpetas, archivos, papelera, compartición, comentarios y administración de usuarios) escribe un registro en `activity_logs` con el usuario, la acción, el tipo y el id del recurso, una copia de su nombre, los detalles propios de la acción y la fecha. La copia del nombre garantiza que el historial siga siendo legible aunque el elemento se elimine de forma definitiva. El registro se hace después de completar la operación y, si fallara, el error solo se escribe en la consola para no impedir la operación principal.

6.7 Reportes

Los doce reportes se calculan con pipelines de agregación de MongoDB sobre las colecciones `files`, `shares` y `activity_logs`, y resuelven los datos de otras colecciones con `$lookup`. El administrador consulta información global; un usuario normal siempre ve únicamente la suya (el servidor ignora o rechaza cualquier intento de consultar a otro usuario). Las fechas de los filtros se interpretan en hora de Guatemala (UTC-6) y el día final se incluye completo; la agrupación por día, semana o mes usa la misma zona horaria. Cada reporte limita la cantidad de filas con un valor por defecto y un máximo.

Reporte	Colección	Operadores principales
Archivos por tipo	`files`	`$match`, `$group`, `$sort`
Archivos en un rango de fechas	`files`	`$match`, `$facet`, `$sort`, `$limit`, `$lookup`
Archivos con más operaciones	`activity_logs`	`$match`, `$group`, `$sort`, `$limit`, `$lookup`
Usuarios con más archivos	`files`	`$group`, `$sort`, `$limit`, `$lookup`
Usuarios con más recursos compartidos	`shares`	Doble `$group` (recurso y propietario), `$sort`, `$lookup`
Archivos compartidos con más usuarios	`shares`	`$match`, `$group`, `$lookup`, `$unwind`
Compartidos por / recibidos por un usuario	`shares`	`$match`, varios `$lookup`, `$addFields`, `$project`
Historial de un usuario	`activity_logs`	`$match`, `$facet`, `$sort`, `$limit`
Operaciones por tipo / principales acciones	`activity_logs`	`$group`, `$sort`, `$facet`
Operaciones por periodo	`activity_logs`	`$group` con `$dateToString` (zona horaria de Guatemala)

7. API REST

Todos los endpoints, salvo el de estado del servidor y el de inicio de sesión, requieren el encabezado `Authorization: Bearer <token>`. La columna Acceso indica el permiso mínimo necesario; "Propietario" significa que el recurso debe pertenecer al usuario, y "Administrador" que solo el rol de administrador puede usarlo. Todas las rutas comienzan con `/api`.

7.1 Estado y autenticación

Método	Ruta	Acceso	Descripción
GET	/api/health	Público	Indica que el servidor está en ejecución.
POST	/api/auth/login	Público	Autentica y devuelve el token JWT y los datos del usuario.
GET	/api/auth/me	Autenticado	Devuelve los datos del usuario de la sesión.

7.2 Usuarios

Método	Ruta	Acceso	Descripción
GET	/api/users?search&role&active	Administrador	Lista usuarios con filtros opcionales.
POST	/api/users	Administrador	Crea un usuario (nombre, correo, contraseña y rol).
PUT	/api/users/:id	Administrador	Edita nombre, rol o contraseña.
PATCH	/api/users/:id/active	Administrador	Activa o desactiva una cuenta.

7.3 Carpetas

Método	Ruta	Acceso	Descripción
GET	/api/folders?parentId	Lectura	Lista las subcarpetas; sin `parentId`, las de la raíz.
POST	/api/folders	Edición	Crea una carpeta (en la raíz o dentro de otra).
GET	/api/folders/:id	Lectura	Devuelve la carpeta, su ruta de navegación y el nivel de acceso.
PATCH	/api/folders/:id/rename	Edición	Renombra la carpeta.
PATCH	/api/folders/:id/move	Propietario	Mueve la carpeta; impide ciclos.

Método	Ruta	Acceso	Descripción
DELETE	/api/folders/:id	Propietario	Envía la carpeta y su contenido a la papelera.

7.4 Archivos y comentarios

Método	Ruta	Acceso	Descripción
GET	/api/files?folderId&q&ext&sort&order	Lectura	Lista y busca archivos con filtros y orden.
POST	/api/files	Edición	Sube un archivo (multipart, campo `file` y `folderId` opcional).
GET	/api/files/:id	Lectura	Datos del archivo y nivel de acceso.
GET	/api/files/:id/content	Lectura	Contenido como texto; registra `file.view`.
GET	/api/files/:id/download	Lectura	Descarga el archivo; registra `file.download`.
PUT	/api/files/:id/content	Edición	Reemplaza el contenido.
PATCH	/api/files/:id/rename	Edición	Renombra (debe terminar en .txt o .md).
PATCH	/api/files/:id/move	Propietario	Mueve el archivo a otra carpeta o a la raíz.
DELETE	/api/files/:id	Propietario	Envía el archivo a la papelera.
GET	/api/files/:id/comments	Lectura	Lista los comentarios del archivo.
POST	/api/files/:id/comments	Comentar	Agrega un comentario (hasta 1000 caracteres).
DELETE	/api/files/:id/comments/:commentId	Lectura	Borra un comentario; solo su autor o el propietario.

7.5 Compartición

Método	Ruta	Acceso	Descripción
POST	/api/shares	Propietario	Comparte un archivo o carpeta con un usuario (correo y permiso).

Método	Ruta	Acceso	Descripción
GET	/api/shares?resourceType&resourceId	Propietario	Con quién está compartido un recurso.
GET	/api/shares/by-me	Autenticado	Lo que el usuario compartió.
GET	/api/shares/with-me	Autenticado	Lo que otros compartieron con el usuario.
PATCH	/api/shares/:id	Quien lo otorgó	Cambia el permiso.
DELETE	/api/shares/:id	Quien lo otorgó	Retira el permiso.

7.6 Papelera

Todas las rutas de la papelera operan únicamente sobre la papelera del propio usuario.

Método	Ruta	Acceso	Descripción
GET	/api/trash	Propietario	Lista lo eliminado directamente, con el contenido de cada carpeta.
POST	/api/trash/files/:id/restore	Propietario	Restaura un archivo.
POST	/api/trash/folders/:id/restore	Propietario	Restaura una carpeta con lo eliminado junto con ella.
DELETE	/api/trash/files/:id	Propietario	Elimina un archivo de forma definitiva.
DELETE	/api/trash/folders/:id	Propietario	Elimina una carpeta y su contenido de forma definitiva.
DELETE	/api/trash	Propietario	Vacía la papelera.

7.7 Reportes

Todos los reportes son de tipo GET y aceptan los filtros que correspondan: `from`, `to`, `limit`, `userId`, `ownerId`, `ext`, `includeTrash`, `permission`, `resourceType`, `action` y `period`.

Ruta	Acceso	Descripción
/api/reports	Autenticado	Catálogo de reportes disponibles según el rol.
/api/reports/files-by-type	Administrador	Cantidad de archivos por tipo.
/api/reports/files-by-date	Administrador	Archivos almacenados en un rango de fechas.
/api/reports/files-most-operations	Administrador	Archivos con más operaciones.
/api/reports/top-users-by-files	Administrador	Usuarios con más archivos.

Ruta	Acceso	Descripción
/api/reports/top-users-by-shares	Administrador	Usuarios con más recursos compartidos.
/api/reports/most-shared-files	Administrador	Archivos compartidos con más usuarios.
/api/reports/shared-by-user	Autenticado	Recursos compartidos por un usuario.
/api/reports/shared-with-user	Autenticado	Recursos recibidos por un usuario.
/api/reports/user-activity	Autenticado	Historial de actividad de un usuario.
/api/reports/operations-by-type	Autenticado	Operaciones agrupadas por tipo.
/api/reports/operations-by-period	Autenticado	Operaciones agrupadas por día, semana o mes.
/api/reports/top-actions	Autenticado	Principales acciones, con porcentaje.

En los reportes marcados como "Autenticado", el administrador puede consultar a cualquier usuario con `userId`; un usuario normal solo puede consultarse a sí mismo y recibe 403 si pide a otro.

7.8 Códigos de respuesta

Todos los errores responden con un cuerpo JSON de la forma `{ "message": "..." }`:

Código	Cuándo se usa
200 / 201	La operación se realizó (201 cuando se crea un recurso).
400	Datos inválidos: campos vacíos, identificadores mal formados, extensión no permitida, filtro o fecha inválidos, ciclo al mover una carpeta.
401	Falta el token, es inválido o venció, la cuenta está desactivada o las credenciales son incorrectas.
403	El usuario no tiene acceso al recurso o su permiso no alcanza para la acción; también el rol no permitido.
404	El recurso no existe o ya está en la papelera, o la ruta no existe.
409	Conflicto: nombre repetido en la misma ubicación, correo ya registrado o recurso ya compartido con esa persona.
413	El archivo o el contenido supera los 5 MB.
500	Error interno; el mensaje es genérico y el detalle queda en la consola del servidor.

8. Frontend

8.1 Arquitectura

La aplicación cliente utiliza componentes standalone de Angular 21, sin módulos (NgModule), el estado de cada pantalla se maneja con signals, la detección de cambios es sin zone.js (el valor por defecto en Angular 21) y las plantillas usan el control de flujo nativo (`@if`, `@for`, `@switch`). La comunicación con la API se realiza con HttpClient y Observables de RxJS. Las pantallas se cargan bajo demanda (lazy loading) y los parámetros de la ruta llegan a los componentes como entradas gracias a `withComponentInputBinding`.

8.2 Carpeta core

Archivo	Contenido
`config.ts`	Dirección del servidor (`API_URL`). Es el único lugar que hay que cambiar si el servidor usa otro puerto.
`models.ts`	Interfaces de TypeScript que reflejan la forma de las respuestas de la API.
`*.service.ts`	Un servicio por módulo (auth, folders, files, comments, trash, shares, users, reports) que realiza las peticiones HTTP.
`interceptors.ts`	Agrega el token a cada petición hacia la API y cierra la sesión si el servidor responde 401.
`guards.ts`	Guardas de rutas: sesión iniciada, solo administrador y pantalla de login solo sin sesión.
`errors.ts` y `toast.service.ts`	Convierten los errores HTTP en mensajes para el usuario y muestran los avisos en pantalla.

8.3 Pantallas y componentes

Pantalla	Ruta	Descripción
Inicio de sesión	/login	Formulario con opción para ver la contraseña.
Mis archivos	/files y /files/:folderId	Explorador: navegar, crear, subir, descargar, renombrar, mover, buscar y enviar a la papelera.
Visor	/view/:fileId	Muestra el archivo (Markdown con formato), permite editarlo y contiene los comentarios.
Compartidos	/shared	Pestañas "conmigo" y "por mi", con filtro, cambio de permiso y retiro de acceso.
Papelera	/trash	Restaurar, eliminar definitivamente y vaciar.
Reportes	/reports	Lista de reportes, filtros, tablas con barras y descarga en CSV.
Usuarios	/users	Administración de cuentas (solo administrador).

Además hay componentes reutilizables en `shared` y en las carpetas de cada pantalla: ventana emergente, avisos, iconos, selector de carpeta destino, ventana de compartir, panel de comentarios y vista de Markdown. El menú lateral (`layout`) muestra la opción Usuarios solo a los administradores.

8.4 Seguridad en el cliente

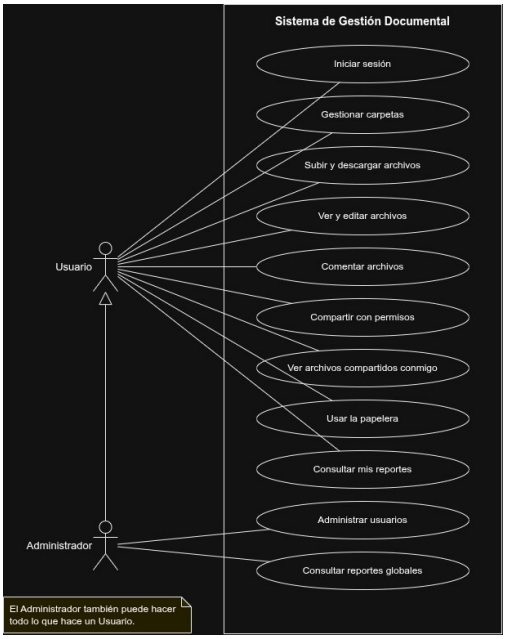
El token JWT y los datos del usuario se guardan en `localStorage`, y el interceptor agrega el token automáticamente a cada petición que va a la API. Al abrir la aplicación se confirma con `GET /api/auth/me` que la sesión siga siendo válida. Las rutas están protegidas por `authGuard` (sesión iniciada) y `adminGuard` (solo administrador), y el visor pide confirmación antes de salir con cambios sin guardar. Estas protecciones son una comodidad de navegación: la seguridad real reside en el servidor, que rechaza con 401 o 403 cualquier petición no autorizada, incluso si alguien accediera a la API sin pasar por la interfaz.

9. Diagramas UML

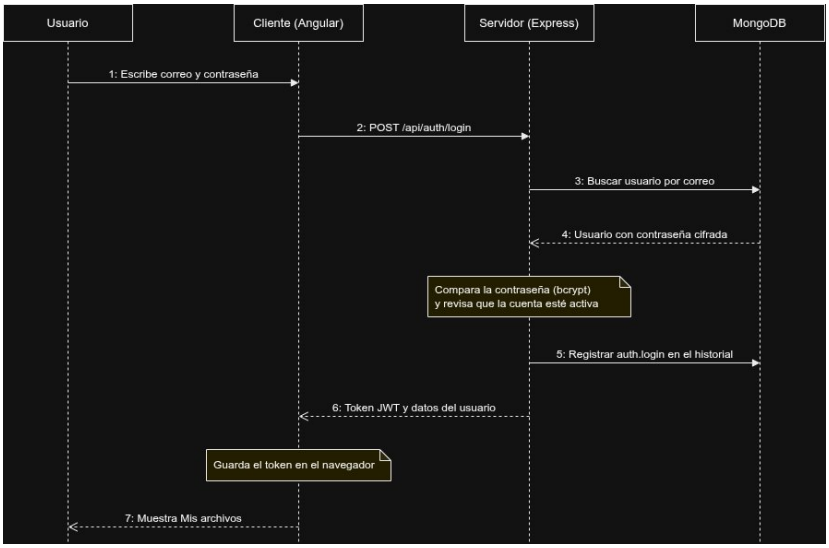
Los siguientes diagramas complementan este manual. Se encuentran en la carpeta `docs/diagramas` del repositorio, en el archivo editable `Diagramas_UML_Proyecto_2.drawio` (elaborado en draw.io), junto con su exportación.

9.1 Diagrama de casos de uso

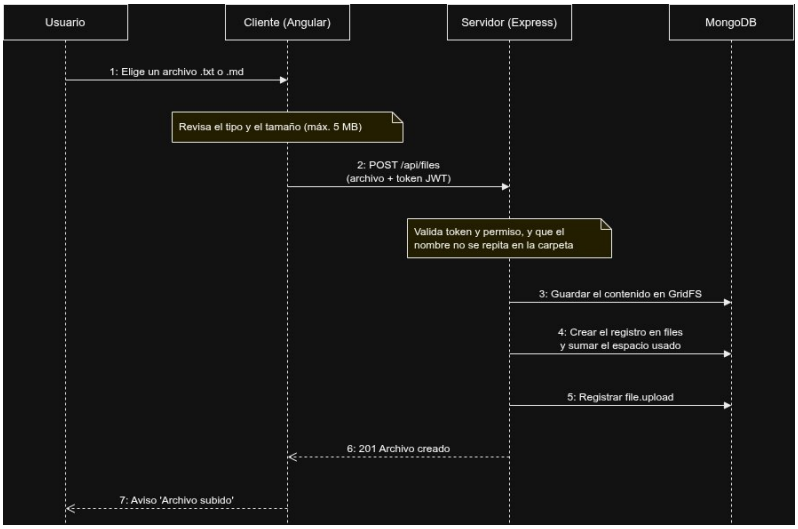
Muestra a los dos actores del sistema, Usuario y Administrador (que hereda todo lo del Usuario), y las funciones que cada uno puede realizar.



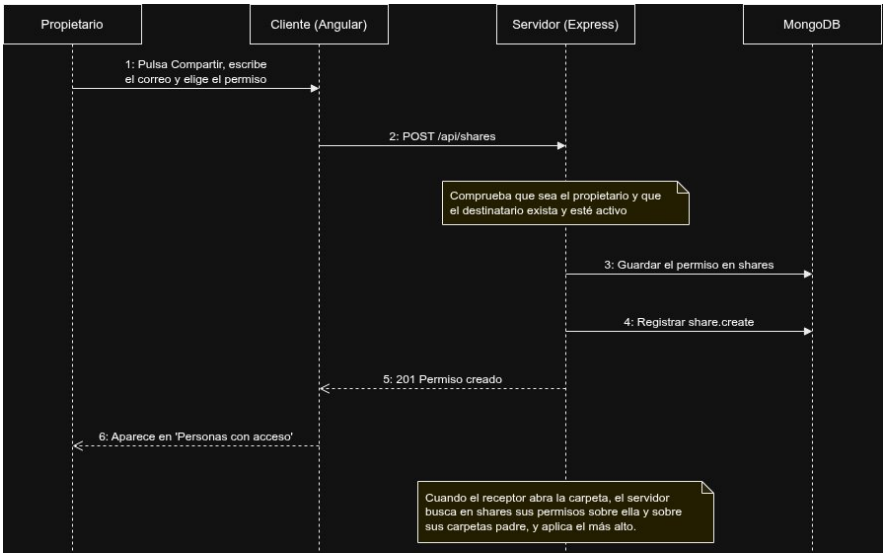
9.2 Diagrama de secuencia: iniciar sesión



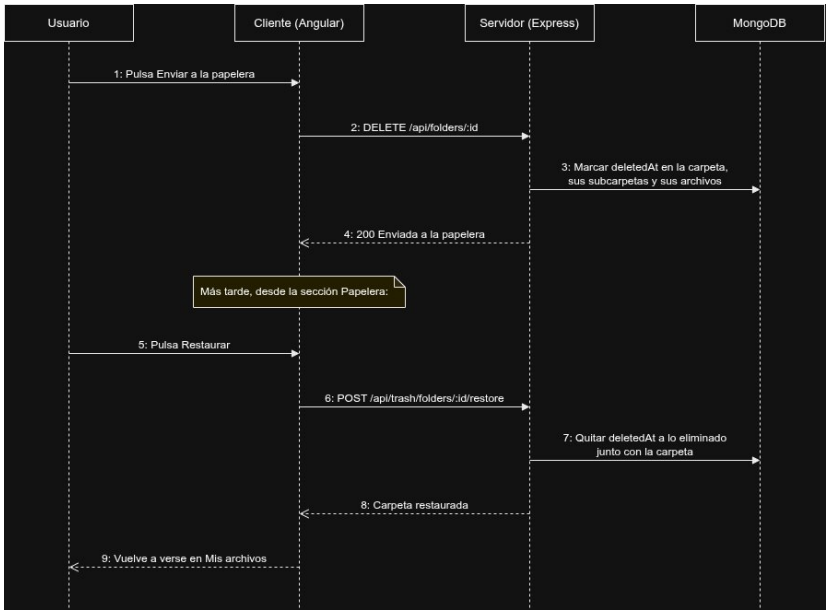
9.3 Diagrama de secuencia: subir un archivo



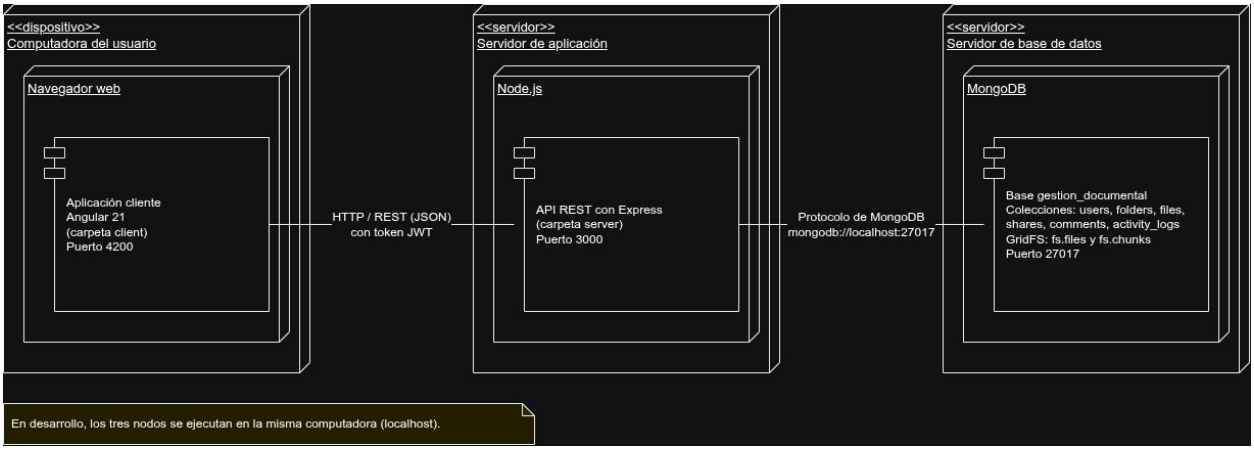
9.4 Diagrama de secuencia: compartir una carpeta



9.5 Diagrama de secuencia: papelera



9.6 Diagrama de despliegue



Muestra los tres nodos del sistema: el navegador con la aplicación Angular, el servidor Node.js con Express y el servidor MongoDB, y los protocolos que los comunican.

10. Pruebas Realizadas

Durante el desarrollo se realizaron pruebas manuales sobre los endpoints (con peticiones HTTP a un servidor real conectado a MongoDB) y sobre la interfaz, verificando en particular:

- Inicio de sesión correcto e incorrecto, cuenta desactivada y ausencia de token (401).
- Restricción por rol: un usuario normal recibe 403 al intentar administrar usuarios o consultar un reporte global.
- Administración de usuarios: crear, editar, desactivar y reactivar, y bloqueo del ingreso de una cuenta desactivada.
- Carpetas: crear, renombrar y mover; rechazo de nombres repetidos y de mover una carpeta dentro de sí misma o de una subcarpeta (400); recálculo de la ruta de las subcarpetas.
- Archivos: subir, leer, descargar y editar el contenido en GridFS; rechazo de extensiones no permitidas (400); y, desde la interfaz, renombrar, mover y buscar.
- Papelera: enviar una carpeta con contenido, restaurarla completa, restaurar un elemento cuya carpeta sigue eliminada (pasa a la raíz), eliminación definitiva con limpieza del contenido y del espacio usado.
- Compartición: cada nivel de permiso contra cada acción (lectura sin poder crear, edición sin poder mover ni eliminar, acceso por herencia a las subcarpetas), y que lo creado por un editor pertenezca al propietario de la carpeta.
- Comentarios: escribir con permiso suficiente, rechazo con solo lectura (403) y borrado por su autor o por el propietario.
- Reportes: los doce reportes con los datos de la carga inicial, la coherencia entre sus totales (por ejemplo, la suma por tipo coincide con la suma por usuario) y el alcance según el rol.
- Flujo completo desde la interfaz con los usuarios de prueba: subir, organizar, compartir con otro usuario, comentar, enviar a la papelera y restaurar.

Figura 8. Espacio para agregar otras pruebas realizadas o capturas de evidencia.

11. Decisiones Técnicas y Justificación

Decisión	Justificación
Permisos en una colección aparte (`shares`) y no embebidos en cada archivo	Los reportes de compartición se resuelven con una sola colección, y compartir o retirar un permiso no modifica el documento del archivo.
Arreglo `ancestors` embebido en cada carpeta	Permite detectar ciclos, heredar permisos y localizar subcarpetas con una sola consulta, sin recorridos recursivos.

Decisión	Justificación
Contenido de los archivos en GridFS	Un documento no puede superar 16 MB; con GridFS el listado trabaja solo con metadatos y el contenido se lee únicamente al abrir o descargar.
Borrado lógico con `deletedAt` y una fecha común para la cascada	Conserva el historial y permite restaurar una carpeta con exactamente lo que se eliminó junto con ella.
Orden de las operaciones en lugar de transacciones	MongoDB local no es un conjunto de réplicas y no admite transacciones; ordenar las escrituras evita estados inconsistentes.
JWT sin sesiones en el servidor, con consulta del usuario en cada petición	Simplifica el servidor y es el estándar para una SPA; la consulta permite bloquear de inmediato a un usuario desactivado.
El administrador no accede al contenido de archivos ajenos	Cumple la regla del enunciado de que solo se accede a los recursos propios o compartidos; el administrador consulta información general en los reportes.
Serialización de respuestas	Evita filtrar datos sensibles, como el hash de la contraseña, y desacopla la API de los esquemas.
Historial con copia del nombre del recurso	El historial debe seguir siendo legible aunque el recurso se elimine de forma definitiva.
Índices creados con un script y `autoIndex` desactivado	La base de datos depende de los scripts del estudiante y no de la configuración de Mongoose.
Express en lugar de Next.js	Solo se necesita una API REST; Express es más simple y suficiente para el alcance del proyecto.
Archivos recibidos en memoria con un máximo de 5 MB	Los archivos son de texto y pequeños; evita archivos temporales en disco y simplifica el guardado en GridFS.
Reportes con agregaciones en hora de Guatemala	Los totales por día y los rangos de fechas coinciden con el día real del usuario; el enunciado pide aprovechar las agregaciones de MongoDB.
Componentes standalone, signals y lazy loading en Angular	Es el enfoque recomendado en Angular 21; reduce el código y carga cada pantalla solo cuando se necesita.
Markdown con la librería `marked` y sanitizador de Angular	Muestra el formato sin permitir que un archivo ajeno ejecute código en el navegador.

12. Instalación para Desarrollo

Los pasos detallados de instalación y ejecución se documentan en el Manual de Usuario, sección 2. Para el desarrollo se recomienda además:

- Iniciar el servidor con `npm run dev`, que lo reinicia automáticamente al guardar cambios.
- Verificar en cada arranque que la consola muestre **Conectado a MongoDB** y **Servidor escuchando** sin errores.
- Si se agrega un índice, definirlo en `database/indexes.js` y volver a ejecutar el script (es repetible).
- No definir índices en los esquemas de Mongoose: la creación automática está desactivada a propósito.
- Ejecutar `node seed.js` para volver a los datos iniciales; esta operación borra la información actual.
- Compilar el cliente para producción con `ng build`, que genera los archivos estáticos en `client/dist/`.
- No subir nunca el archivo `server/.env` al repositorio.

13. Conclusiones

El Sistema de Gestión Documental cumple con los requerimientos funcionales y técnicos establecidos: arquitectura cliente-servidor con aplicaciones independientes, base de datos documental con contenido de archivos en GridFS, control de acceso por rol y por recurso con herencia, papelera con restauración consistente, historial de actividad y un módulo de reportes basado en agregaciones.

El diseño en capas, la separación entre backend y frontend y la documentación del modelo permiten que el sistema pueda mantenerse y ampliarse de forma ordenada, agregando nuevos módulos sin comprometer la lógica ya implementada.